

DeploySage

AI-powered CI/CD intelligence for pull requests and deployment safety

Project Definition + Technical Requirements Document

A containerized AI DevOps platform that integrates with GitHub Actions to auto-generate PR documentation, diagnose CI/CD failures, score deployment risk, enforce PR-based release gates, and maintain versioned analysis history.

Top 5 Features

#	Feature	Description
1	AI-generated PR documentation	Analyzes code diffs and generates PR summaries, key changes, testing notes, deployment notes, and risk factors.
2	CI/CD failure diagnosis	Ingests failed GitHub Actions logs, extracts relevant error context, and uses LLMs to identify root causes.
3	Deployment risk scoring	Scores each PR using CI status, changed files, dependency updates, config changes, and critical code paths.
4	PR-based release gates	Posts risk reports and status checks directly on pull requests to warn or block high-risk deployments.
5	Versioned analysis history	Tracks every generated PR summary, failure diagnosis, risk score, prompt version, model version, and deployment decision across PR updates.

1. High-level Project Explanation

What is DeploySage?

DeploySage is a containerized AI DevOps platform that integrates with GitHub Actions and pull requests to make CI/CD pipelines smarter. Whenever a developer opens or updates a pull request, DeploySage analyzes the code diff, generates PR documentation, scores deployment risk, and later diagnoses CI/CD failures if the workflow breaks.

In simple terms: DeploySage is like an AI DevOps reviewer sitting inside the CI/CD pipeline. It does not replace engineers. It helps engineers understand what changed, why a build failed, and whether a release is safe.

Why this project matters

- Most CI/CD systems tell developers what failed, but not clearly why it failed.
- PR descriptions are often incomplete, inconsistent, or written after the fact.
- Deployment risk is usually judged manually instead of being surfaced directly inside the PR.
- LLMs are useful here because they can summarize diffs, explain logs, and generate readable operational context.
- The project demonstrates backend engineering, DevOps automation, LLM integration, and platform thinking together.

High-level workflow

```
Developer opens or updates PR
↓
GitHub Actions workflow runs
↓
DeploySage receives PR metadata + code diff
↓
LLM generates PR documentation
↓
DeploySage updates PR body / posts comment
↓
CI tests run
↓
If CI fails, logs are sent to DeploySage
↓
DeploySage extracts useful error chunks
↓
LLM generates root-cause summary + fix recommendation
↓
DeploySage calculates deployment risk score
↓
DeploySage posts status check/comment on PR
↓
All analyses are versioned and stored
```

Resume-ready project definition

One-line description:

DeploySage is an AI-powered DevOps platform that integrates with GitHub Actions to auto-generate PR documentation, diagnose CI/CD failures, score deployment risk, enforce PR-based release gates, and maintain versioned analysis history.

Resume bullet:

Developed DeploySage, a containerized AI DevOps platform using GitHub Actions, Docker, PostgreSQL, and LLM APIs to automate PR documentation, analyze CI/CD failures, generate root-cause summaries, score deployment risk, and enforce release gates through PR checks.

2. Feature Scope

Feature 1: AI-generated PR documentation

DeploySage analyzes the PR diff and generates summary, key changes, files affected, testing notes, deployment notes, and risk factors. It should update only a marked AI section inside the PR body instead of blindly overwriting the developer-written description.

Feature 2: CI/CD failure diagnosis

When GitHub Actions fails, DeploySage receives raw logs, extracts useful lines around errors, exceptions, timeouts, missing modules, permissions, and test failures, then asks the LLM to produce root cause, suggested fix, failure type, confidence, and evidence lines.

Feature 3: Deployment risk scoring

DeploySage gives every PR a risk score from 0 to 100. The numeric score should be calculated using rule-based signals, while the LLM explains the score in human language. This keeps the system trustworthy and engineering-heavy.

Feature 4: PR-based release gates

DeploySage posts PR comments or status checks. Low-risk PRs can pass, medium-risk PRs can warn, and high-risk PRs can require manual review or block deployment.

Feature 5: Versioned analysis history

Every time a PR changes, DeploySage stores a new analysis version with commit SHA, prompt version, model version, PR summary, risk score, failure diagnosis, and deployment gate decision.

Example PR documentation output

```
## Summary
This PR adds JWT-based authentication middleware and protects the log-analysis API.

## Key Changes
- Added authMiddleware.ts for token validation
- Updated /analyze-log route to require authorization
- Added unit tests for invalid and missing tokens

## Testing
- Added tests for protected routes
- Existing API tests pass locally

## Deployment Notes
- Ensure JWT_SECRET is configured in production secrets
```

Example risk report output

```
Risk Score: 74/100
Risk Level: HIGH
Signals:
- Authentication middleware changed
- package-lock.json changed
- 2 backend tests failed
- New environment variable required
Recommendation: Require manual approval before deployment.
```

3. Technical Requirements Document

3.1 System Goals

- Integrate with GitHub pull requests and GitHub Actions.
- Generate PR documentation from code diffs.
- Analyze failed CI/CD logs and produce root-cause summaries.
- Calculate deployment risk scores using rule-based signals.
- Post comments or checks back to GitHub PRs.
- Store every analysis version in PostgreSQL.
- Run as a Dockerized backend service.
- Have its own CI/CD pipeline.

Non-goals for MVP

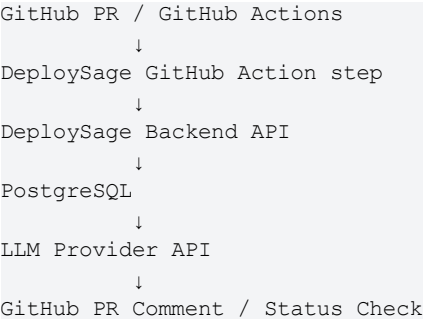
- Full GitHub App marketplace integration.
- Multi-organization SaaS support.
- Kubernetes deployment.
- Complex frontend dashboard.
- Slack bot.
- Self-healing auto-PRs.
- Vector database.

- Fine-tuned LLM.

3.2 Users

User Type	Need
Developer	Understand what changed, why CI failed, and what needs to be fixed.
Team Lead	Review risk, deployment impact, and whether a PR should be merged.
DevOps / Platform Engineer	Enforce release safety and reduce repetitive CI/CD troubleshooting.

3.3 System Architecture



Recommended Tech Stack

Layer	Recommended Choice
Backend	Node.js + Express + TypeScript
Database	PostgreSQL
ORM	Prisma
CI/CD	GitHub Actions
Containerization	Docker
LLM	Hugging Face Inference API or OpenAI API
Testing	Jest
Validation	Zod
Deployment	Render, Railway, Fly.io, or AWS ECS

4. Core Backend Modules

```

src/
  app.ts
  server.ts
  config/
    env.ts
  routes/
    pr.routes.ts
    analysis.routes.ts
    risk.routes.ts
    github.routes.ts
    health.routes.ts
  controllers/
    pr.controller.ts
    analysis.controller.ts
    risk.controller.ts
  services/
    github.service.ts
    llm.service.ts
    prSummary.service.ts

```

```

    logAnalysis.service.ts
    riskScoring.service.ts
    versioning.service.ts
  utils/
    logExtractor.ts
    diffParser.ts
    promptBuilder.ts
    githubCommentFormatter.ts
  prisma/
    schema.prisma
  tests/
    riskScoring.test.ts
    logExtractor.test.ts
    prSummary.test.ts

```

Service Responsibilities

Service	Responsibilities
prSummary.service.ts	Build prompt from diff, call LLM, parse JSON, format markdown, store PR summary version.
logAnalysis.service.ts	Receive raw logs, extract useful error chunks, call LLM, store failure analysis version.
riskScoring.service.ts	Apply weighted risk rules, generate risk score, call LLM for explanation, store risk score version.
github.service.ts	Fetch metadata, update PR body, post comments, create/update status checks.
versioning.service.ts	Find latest analysis version, increment version, attach commit SHA, retrieve history.

5. Database Design

Use PostgreSQL. The core idea is that every PR update creates a new analysis run, and each analysis run can contain a PR summary version, failure analysis version, risk score version, and deployment gate decision.

Main entities

```

Repository
PullRequest
AnalysisRun
PRSummaryVersion
FailureAnalysisVersion
RiskScoreVersion
PromptVersion
DeploymentGateDecision

```

repositories

```

CREATE TABLE repositories (
  id UUID PRIMARY KEY,
  github_owner TEXT NOT NULL,
  github_repo TEXT NOT NULL,
  default_branch TEXT,
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW()
);

```

pull_requests

```

CREATE TABLE pull_requests (
  id UUID PRIMARY KEY,
  repository_id UUID REFERENCES repositories(id),
  pr_number INT NOT NULL,

```

```

title TEXT,
author TEXT,
base_branch TEXT,
head_branch TEXT,
latest_commit_sha TEXT,
state TEXT,
created_at TIMESTAMP DEFAULT NOW(),
updated_at TIMESTAMP DEFAULT NOW()
);

```

analysis_runs

```

CREATE TABLE analysis_runs (
  id UUID PRIMARY KEY,
  pull_request_id UUID REFERENCES pull_requests(id),
  commit_sha TEXT NOT NULL,
  workflow_run_id TEXT,
  analysis_version INT NOT NULL,
  status TEXT NOT NULL,
  model_name TEXT,
  prompt_version TEXT,
  created_at TIMESTAMP DEFAULT NOW()
);

```

pr_summary_versions

```

CREATE TABLE pr_summary_versions (
  id UUID PRIMARY KEY,
  analysis_run_id UUID REFERENCES analysis_runs(id),
  summary_markdown TEXT NOT NULL,
  key_changes JSONB,
  testing_notes TEXT,
  deployment_notes TEXT,
  created_at TIMESTAMP DEFAULT NOW()
);

```

failure_analysis_versions

```

CREATE TABLE failure_analysis_versions (
  id UUID PRIMARY KEY,
  analysis_run_id UUID REFERENCES analysis_runs(id),
  failure_type TEXT,
  root_cause TEXT,
  suggested_fix TEXT,
  confidence NUMERIC,
  raw_log_excerpt TEXT,
  cleaned_log_excerpt TEXT,
  created_at TIMESTAMP DEFAULT NOW()
);

```

risk_score_versions

```

CREATE TABLE risk_score_versions (
  id UUID PRIMARY KEY,
  analysis_run_id UUID REFERENCES analysis_runs(id),
  risk_score INT NOT NULL,
  risk_level TEXT NOT NULL,
  signals JSONB,
  explanation TEXT,
  recommendation TEXT,
  created_at TIMESTAMP DEFAULT NOW()
);

```

deployment_gate_decisions

```
CREATE TABLE deployment_gate_decisions (  
  id UUID PRIMARY KEY,  
  analysis_run_id UUID REFERENCES analysis_runs(id),  
  decision TEXT NOT NULL,  
  reason TEXT,  
  created_at TIMESTAMP DEFAULT NOW()  
);
```

6. API Design

Method	Endpoint	Purpose
GET	/health	Check whether the API is alive.
POST	/api/pr/analyze	Analyze PR metadata, diff, changed files, and CI status.
POST	/api/ci/analyze-failure	Analyze failed CI/CD logs and generate root cause + fix.
GET	/api/pr/owner/:repo/:prNumber/history	Return versioned analysis history for a PR.

POST /api/pr/analyze request example

```
{  
  "repository": { "owner": "chieftecho99", "name": "sample-backend" },  
  "pull_request": {  
    "number": 12,  
    "title": "Add JWT authentication",  
    "author": "chieftecho99",  
    "base_branch": "main",  
    "head_branch": "feature/auth",  
    "commit_sha": "abc123"  
  },  
  "diff": "diff --git a/src/auth.ts b/src/auth.ts ...",  
  "changed_files": ["src/authMiddleware.ts", "src/routes/analyze.ts", "package-lock.json"],  
  "test_status": "passed"  
}
```

POST /api/pr/analyze response example

```
{  
  "analysis_run_id": "uuid",  
  "summary": "This PR adds JWT authentication...",  
  "risk_score": 63,  
  "risk_level": "MEDIUM",  
  "gate_decision": "WARN"  
}
```

POST /api/ci/analyze-failure response example

```
{  
  "failure_type": "dependency_error",  
  "root_cause": "jsonwebtoken is imported but not installed.",  
  "suggested_fix": "Run npm install jsonwebtoken and commit package files.",  
  "confidence": 0.88  
}
```

7. GitHub Actions Integration

PR analysis workflow

```
name: DeploySage PR Analysis

on:
  pull_request:
    types: [opened, synchronize, reopened]

permissions:
  contents: read
  pull-requests: write
  checks: write

jobs:
  analyze-pr:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Get PR diff
        run: |
          git fetch origin ${GITHUB_BASE_REF}
          git diff origin/${GITHUB_BASE_REF}...HEAD > pr.diff

      - name: Send PR diff to DeploySage
        run: |
          curl -X POST "${DEPLOY_SAGE_API_URL}/api/pr/analyze" \
            -H "Content-Type: application/json" \
            -H "Authorization: Bearer ${DEPLOY_SAGE_API_KEY}" \
            -d @payload.json
```

CI failure workflow

```
name: CI

on:
  pull_request:

permissions:
  contents: read
  pull-requests: write
  checks: write

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Install dependencies
        run: npm ci

      - name: Run tests
        run: npm test 2>&1 | tee test.log

      - name: Send failure logs to DeploySage
        if: failure()
        run: |
```



```
curl -X POST "${{ secrets.DEPLOY_SAGE_API_URL }}/api/ci/analyze-failure" \
-H "Content-Type: application/json" \
-H "Authorization: Bearer ${{ secrets.DEPLOY_SAGE_API_KEY }}" \
-d @failure-payload.json
```

8. LLM Prompt Design

PR summary prompt

You are DeploySage, an AI DevOps assistant.

Given the following pull request diff and changed files, generate a clean PR documentation section.

Return valid JSON only.

Required fields:

- summary
- key_changes
- testing_notes
- deployment_notes
- risk_factors

Rules:

- Do not invent tests that are not present.
- Do not claim deployment safety unless the data supports it.

CI failure diagnosis prompt

You are DeploySage, an AI CI/CD failure diagnosis assistant.

Analyze the following GitHub Actions log excerpt.

Return valid JSON only.

Required fields:

- failure_type
- root_cause
- suggested_fix
- confidence
- evidence_lines

Rules:

- Do not guess beyond the logs.
- If the root cause is unclear, say unclear.
- Keep suggested_fix actionable.

Risk explanation prompt

You are DeploySage, an AI DevOps risk explanation assistant.

Given the risk score and signals, explain why this PR has this risk level.

Return valid JSON only.

Required fields:

- explanation
- recommendation

9. Risk Scoring Design

Signal	Weight
--------	--------

CI failed	+30
Auth code changed	+20
Payment/billing code changed	+20
Database migration changed	+18
Dependency lock file changed	+12
Dockerfile changed	+12
GitHub Actions workflow changed	+12
Environment/config file changed	+10
More than 20 files changed	+10
Test files added/updated	-8
CI passed	-20

Score Range	Risk Level
0-30	LOW
31-65	MEDIUM
66-100	HIGH

Example risk scoring logic

```
function calculateRiskScore(input: RiskInput): RiskResult {
  let score = 0;
  const signals: string[] = [];

  if (input.ciStatus === "failed") {
    score += 30;
    signals.push("CI failed");
  }

  if (input.changedFiles.some(file => file.includes("auth"))) {
    score += 20;
    signals.push("Authentication-related code changed");
  }

  if (input.changedFiles.some(file => file.includes("migration"))) {
    score += 18;
    signals.push("Database migration changed");
  }

  if (input.changedFiles.includes("package-lock.json")) {
    score += 12;
    signals.push("Dependency lock file changed");
  }

  if (input.changedFiles.some(file => file.includes(".github/workflows"))) {
    score += 12;
    signals.push("CI/CD workflow changed");
  }

  if (input.changedFiles.length > 20) {
    score += 10;
    signals.push("Large PR with more than 20 files changed");
  }

  if (input.changedFiles.some(file => file.includes(".test.") || file.includes(".spec."))) {
    score -= 8;
    signals.push("Tests were added or updated");
  }

  if (input.ciStatus === "passed") {
    score -= 20;
  }
}
```

```

    signals.push("CI passed");
  }

  score = Math.max(0, Math.min(100, score));

  const riskLevel = score >= 66 ? "HIGH" : score >= 31 ? "MEDIUM" : "LOW";
  return { score, riskLevel, signals };
}

```

10. PR Comment Formats

AI-generated PR description section

```

<!-- deploysage:start -->

## DeploySage PR Summary

### Summary
This PR adds JWT-based authentication middleware and protects the log analysis endpoint.

### Key Changes
- Added auth middleware
- Protected /api/analyze-log
- Added dependency updates

### Testing Notes
No test result was provided at PR analysis time.

### Deployment Notes
Add JWT_SECRET to production secrets before deployment.

<!-- deploysage:end -->

```

Risk report comment

```

## DeploySage Risk Report

Risk Level: HIGH
Risk Score: 74/100
Gate Decision: MANUAL REVIEW REQUIRED

Signals:
- Authentication-related code changed
- Dependency lock file changed
- CI failed

Recommendation:
Fix CI failures and require manual approval before deployment.

```

CI failure diagnosis comment

```

## DeploySage CI Failure Diagnosis

Failure Type: Missing Dependency
Confidence: 87%

Root Cause:
The project imports jsonwebtoken, but the dependency is missing from package.json.

Suggested Fix:
npm install jsonwebtoken

```

Evidence:
Error: Cannot find module 'jsonwebtoken'

11. Security Requirements

- Every GitHub Action call to DeploySage must include Authorization: Bearer DEPLOY_SAGE_API_KEY.
- Reject missing or invalid API keys.
- Store secrets only in GitHub Secrets or deployment environment variables.
- Redact API keys, tokens, passwords, database URLs, JWT secrets, AWS keys, and private keys before sending logs to the LLM.
- Do not commit .env files or credentials.

12. Testing Plan

Test Type	What to Test
Unit tests	Risk scoring logic, log extraction, diff parsing, markdown formatting, version incrementing, secret redaction.
Integration tests	PR analyze endpoint, CI failure endpoint, database writes, mock LLM response parsing, GitHub comment formatter.
End-to-end demo	Sample repo PR opens, DeploySage generates PR summary, tests fail, DeploySage posts diagnosis and risk report.

13. Docker Setup

Dockerfile

```
FROM node:20-alpine

WORKDIR /app

COPY package*.json ./
RUN npm ci

COPY . .

RUN npm run build

EXPOSE 8080

CMD ["npm", "start"]
```

docker-compose.yml

```
version: "3.9"

services:
  api:
    build: .
    ports:
      - "8080:8080"
    environment:
      DATABASE_URL: postgres://postgres:postgres@db:5432/deloysage
      LLM_API_KEY: ${LLM_API_KEY}
      DEPLOY_SAGE_API_KEY: ${DEPLOY_SAGE_API_KEY}
    depends_on:
      - db
```

```

db:
  image: postgres:16
  environment:
    POSTGRES_USER: postgres
    POSTGRES_PASSWORD: postgres
    POSTGRES_DB: deploysage
  ports:
    - "5432:5432"

```

14. CI/CD Pipeline for DeploySage Itself

```

name: DeploySage API CI

on:
  pull_request:
  push:
    branches: [main]

jobs:
  test-build:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4

      - name: Setup Node
        uses: actions/setup-node@v4
        with:
          node-version: 20

      - name: Install dependencies
        run: npm ci

      - name: Run lint
        run: npm run lint

      - name: Run tests
        run: npm test

      - name: Build
        run: npm run build

      - name: Build Docker image
        run: docker build -t deploysage-api .

```

15. MVP Implementation Plan

Phase	Scope
Phase 1: Local backend MVP	Express server, PostgreSQL, Prisma schema, /api/pr/analyze, /api/ci/analyze-failure, mock LLM, risk scoring, versioned storage.
Phase 2: LLM integration	Hugging Face/OpenAI API calls, prompt builder, JSON parser, fallback handling, prompt version tracking.
Phase 3: GitHub Actions integration	Sample repo workflow to collect diff/logs and send payloads to DeploySage.
Phase 4: PR comments and gates	Update PR body between markers, post risk report, post failure diagnosis, create status checks.
Phase 5: Resume polish	README, architecture diagram, demo screenshots/GIF, API docs, Docker Compose, CI badge, deployment link.

Recommended build order

1. Backend API + database
2. Risk scoring engine
3. LLM PR summary generation
4. LLM CI failure diagnosis
5. GitHub Actions integration
6. PR comments and release gates
7. Versioned history endpoint
8. Docker + deployment
9. README + demo

16. Suggested Repository Structure

```
deploysage/  
  README.md  
  docker-compose.yml  
  Dockerfile  
  .github/  
    workflows/  
      ci.yml  
  
apps/  
  api/  
    src/  
      app.ts  
      server.ts  
      routes/  
      controllers/  
      services/  
      utils/  
      config/  
    prisma/  
      schema.prisma  
    tests/  
    package.json  
    tsconfig.json  
  
examples/  
  sample-node-app/  
    .github/  
      workflows/  
        deploysage-pr-analysis.yml  
        ci.yml  
    src/  
    package.json  
  
docs/  
  architecture.md  
  api-reference.md  
  prompt-design.md  
  risk-scoring.md  
  demo-flow.md
```

17. Demo Scenario

A strong demo should show the full lifecycle of one PR.

```
Scenario:
PR #18: Add authentication to log analysis endpoint

Changed files:
- src/middleware/auth.ts
- src/routes/analyze.ts
- package-lock.json
- .env.example

DeploySage output:
- Generates PR summary
- Scores risk as HIGH because auth, dependency, and env config changed
- CI fails due to missing jsonwebtoken
- DeploySage diagnoses missing dependency
- Developer fixes dependency
- New analysis version lowers risk to MEDIUM
```

18. README Checklist

- Project overview and tagline.
- Architecture diagram.
- Top 5 features.
- Tech stack.
- Local setup instructions.
- API examples.
- GitHub Actions integration guide.
- Demo screenshots or GIF.
- Risk scoring rules.
- Database schema summary.
- Future improvements.

19. Final Scope and Difficulty

Version	Difficulty	Meaning
MVP	5/10	Local backend, mock/real LLM, risk scoring, versioned DB storage.
Resume-grade	7.5/10	GitHub Actions integration, PR comments, Docker, polished README, demo.
Production SaaS	9/10	GitHub App, auth, multi-repo/org support, queues, rate limits, dashboard, security hardening.

Final project title

DeploySage - AI-powered CI/CD intelligence for pull requests and deployment safety